

CS202: Software Tools and Techniques for CSE

Lecture 2

Shouvick Mondal

shouvick.mondal@iitgn.ac.in
January 2025

My Group's Contribution to the OSS community

Minecraft: Automated Mining of Software Bug Fixes with Precise Code Context

Sai Krishna Avula
Computer Science and Engineering
IIT Gandhinagar, Gujarat, India
avulasai.krishna@iitgn.ac.in

Venkatesh Vobbliseti
Metallurgical and Materials Engineering
NIT Raipur, Chhattisgarh, India
venkateshseti1211@gmail.com

Shourvik Mondal*
Computer Science and Engineering
IIT Gandhinagar, Gujarat, India
shourvik.mondal@iitgn.ac.in

MineCPP: Mining Bug Fix Pairs and Their Structures

Sai Krishna Avula
IIT Gandhinagar
Gandhinagar, India
avulasai.krishna@iitgn.ac.in

Shourvik Mondal
IIT Gandhinagar
Gandhinagar, India
shourvik.mondal@iitgn.ac.in

program-repair.org

Community-driven effort to facilitate discovery, access and systematization of data related to automated program repair research

Our dataset has been included as a multilingual program repair benchmark in the area of Automated Program Repair (APR).

Bibliography

[AutoCodeRover: Autonomous Program Improvement](#)
Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, Abhik Roychoudhury
ISSTA 2024

[Benchmarking Automated Program Repair: An Extensive Study on Both Real-World and Artificial Bugs](#)
Yicheng Ouyang, Jun Yang, Lingming Zhang
ISSTA 2024

[Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for \\$0.42 Each using ChatGPT](#)
Chunqiu Steven Xia, Lingming Zhang
ISSTA 2024

[BRAFAAR: Bidirectional Refactoring, Alignment, Fault Localization, and Repair for Programming Assignments](#)
Linna Xie, Chongmin Li, Yu Pei, Tian Zhang, Minxue Pan
ISSTA 2024

[CREF: An LLM-Based Conversational Software Repair Framework for Programming Tutors](#)
Boyang Yang, Haoye Tian, Weiguo Pian, Haoran Yu, Haitao Wang, Jacques Klein, Tegawendé F. Bissyandé, Shunfu Jin
ISSTA 2024

[ThinkRepair: Self-Directed Automated Program Repair](#)
Xin Yin, Chao Ni, Shaohua Wang, Zhenhao Li, Limin Zeng, Xiaohu Yang
ISSTA 2024

[View all »](#)

Tools

[ExtractFix](#) — repairs program vulnerabilities via crash constraint extraction

[Gin](#) — a tool for experimentation with GI

[PyGGI](#) — a Python general framework for genetic improvement

[View all »](#)

Benchmarks

[TutorCode](#) — 1,239 C++ buggy codes incorporating human tutor guidance and solution descriptions, accessible via API

[Minecraft](#) — a benchmark with C/C++, Java, and Python programs constructed via automated mining of software bug fixes with precise code context

[BugsPHP](#) — a dataset for automated program repair in PHP

[View all »](#)

Pages

[Defects4J Dissection](#) — presents data to help researchers and practitioners to better understand the Defects4J bug dataset

[RepairThemAll experiment](#) — presents experimental data obtained using RepairThemAll framework

[View all »](#)

Benchmarks

[TutorCode](#) — 1,239 C++ buggy codes incorporating human tutor guidance and solution descriptions, accessible via API

[Minecraft](#) — a benchmark with C/C++, Java, and Python programs constructed via automated mining of software bug fixes with precise code context

[BugsPHP](#) — a dataset for automated program repair in PHP

[View all »](#)

My Group's Contribution to the OSS community

Publicly Available Virtual Machine (sandbox) from SET Group – IITGN.

Provides a playground without worrying about the consequence if anything breaks!

The screenshot shows a Zenodo record page for a Virtual Machine. The browser address bar shows the URL `zenodo.org/records/10467159`. The Zenodo header includes a search bar, navigation links for 'Communities' and 'My dashboard', and buttons for 'Log in' and 'Sign up'. A notice states that Zenodo's user support line is staffed on regular business days between Dec 23 and Jan 5. The record is titled 'Virtual Machine for Software Engineering and Testing Group - IIT Gandhinagar' and was published on January 7, 2024, as version v1. It is categorized as 'Software' and is 'Open'. The author is listed as 'Mondal, Shouvick (Supervisor)'. The affiliations section lists '1. IIT Gandhinagar'. The 'System (host) requirements' section lists hardware and software requirements. The statistics sidebar shows 170 views and 75 downloads. The versions section shows 'Version v1' from January 7, 2024.

zenodo Search records... Communities My dashboard Log in Sign up

Info: Zenodo's user support line is staffed on regular business days between Dec 23 and Jan 5. Response times may be slightly longer than normal.

Published January 7, 2024 | Version v1 Software Open

Virtual Machine for Software Engineering and Testing Group - IIT Gandhinagar

Mondal, Shouvick (Supervisor)¹ Hide affiliations

1. IIT Gandhinagar

System (host) requirements:

Hardware (minimum)

- RAM: 8GB
- #CPUs: 8
- Disk space: 80 GB

Software (minimum)

- Oracle VirtualBox (preferably version 6.1.30)
- Host OS: Windows/Linux/Mac OS

Statistics

	All versions	This version
Views	170	170
Downloads	75	75
Data volume	2.0 TB	2.0 TB

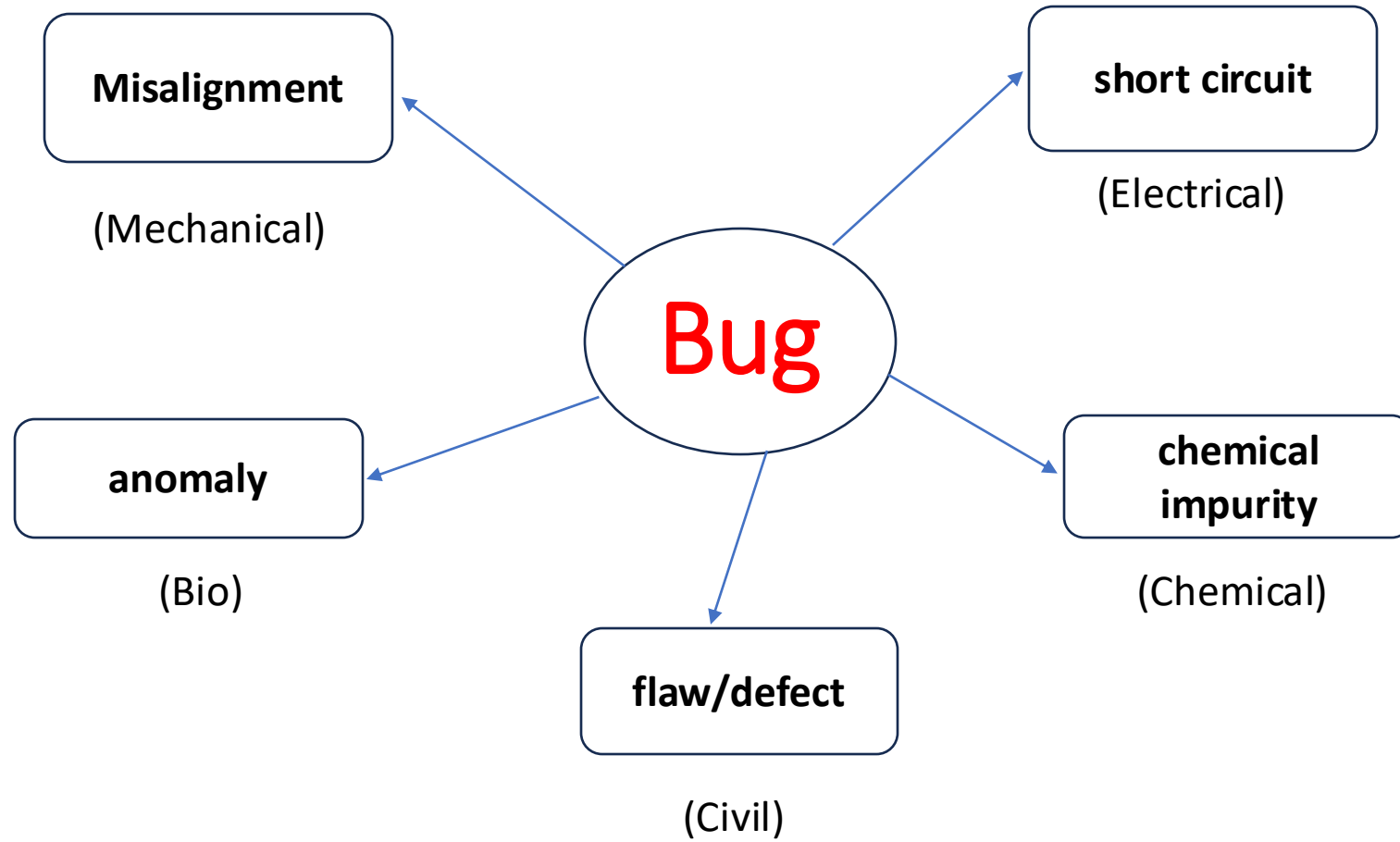
[More info on how stats are collected...](#)

Versions

Version	Published
Version v1	Jan 7, 2024

10.5281/zenodo.10467159

Analogy of Bug



Bug Mining

- Bug mining (bug extraction) – identifying, **extracting code snippets** with **bugs** along with their **corresponding fixes** and **contextual information** from software repositories.
- **Identification** and **resolution** of bugs - critical for **software reliability, performance, and security**.
- Essential for improving software quality, enhancing developer productivity, and advancing our understanding of software defects

The challenges of extracting bug and its contextual information

- **Granularity** of repository-mined bug-fixing datasets/bug-mining techniques
 - **Function level**, **without** precise bug location and bug types.
- **No** language agnostic algorithm to extract code snippets
- *Major issue*: bug-fix context has never been explored...

- **No unified** tools - bug contextual information
- **Unavailability** of GUI to visualise and analyse the bugs

Minecraft: Automated Mining of Software Bug Fixes with Precise Code Context

Published in ASE 2023 (CORE A*) Industry challenge track

<https://www.doi.org/10.1109/ASE56229.2023.00116>

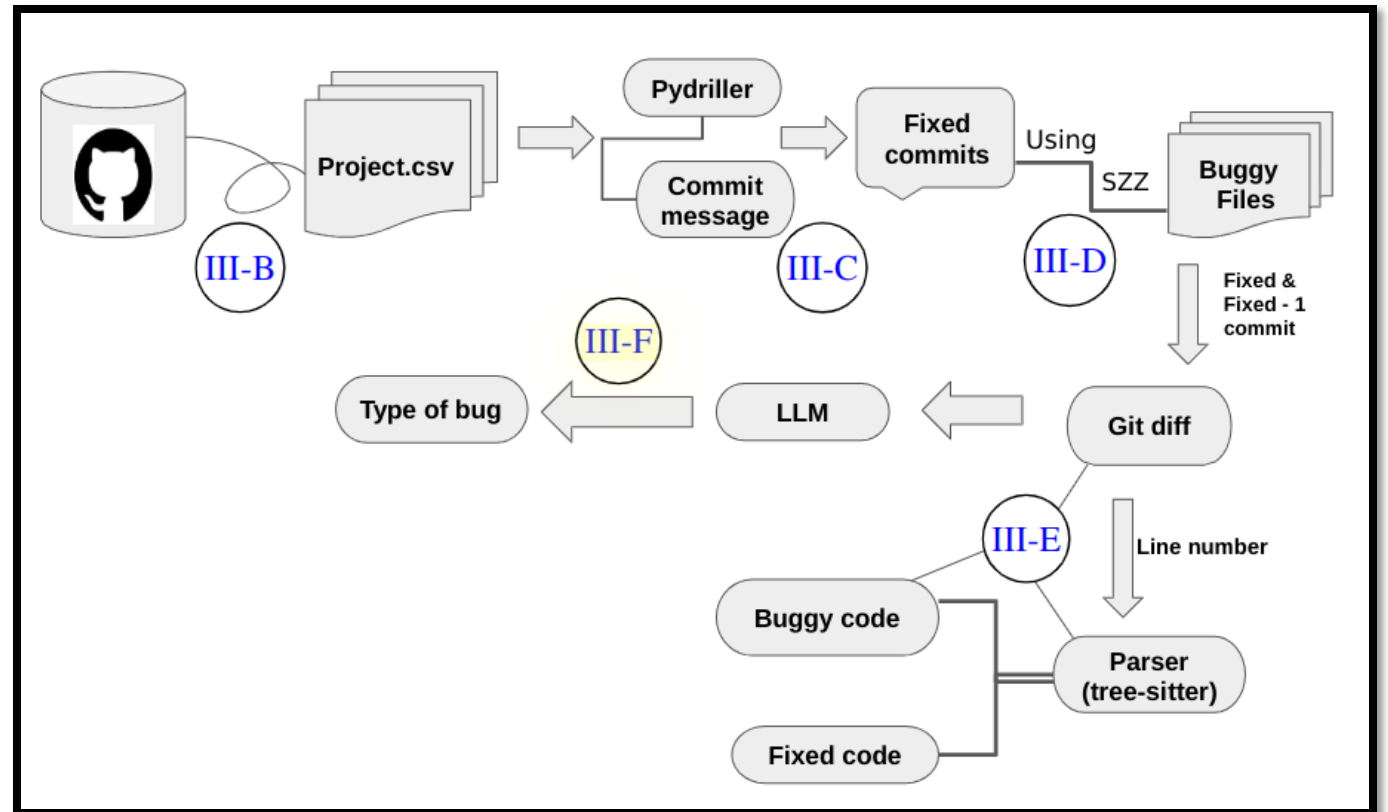
Terminology

- **GitHub** is a **web-based platform** commonly used for **version control** and **collaboration** in software development projects.
- A **commit** refers to a specific **revision** or **snapshot of a project's files** and directories. When a developer makes changes to the codebase, they create a commit to record those changes. A commit typically includes a **concise message** that **describes the purpose or nature of the changes made** - **Commit Message**

Bug Fix Dataset Acquisition Flowchart with Precise Context

Data acquisition is done in following ways:

- **Project Selection**
- Bug fix commits
- Extracting Bug and fixed code Snippets
- Type of the bug



Projects Selection Criteria (Visual Representation)

*In our project selection process, we implemented a rigorous and stringent set of filtering criteria. These criteria were designed to sift through a wide array of options and zero in on real-world projects. This approach ensured that the **projects we selected were not only substantial but also highly relevant to practical applications in the fields of C, C++, and Python.***

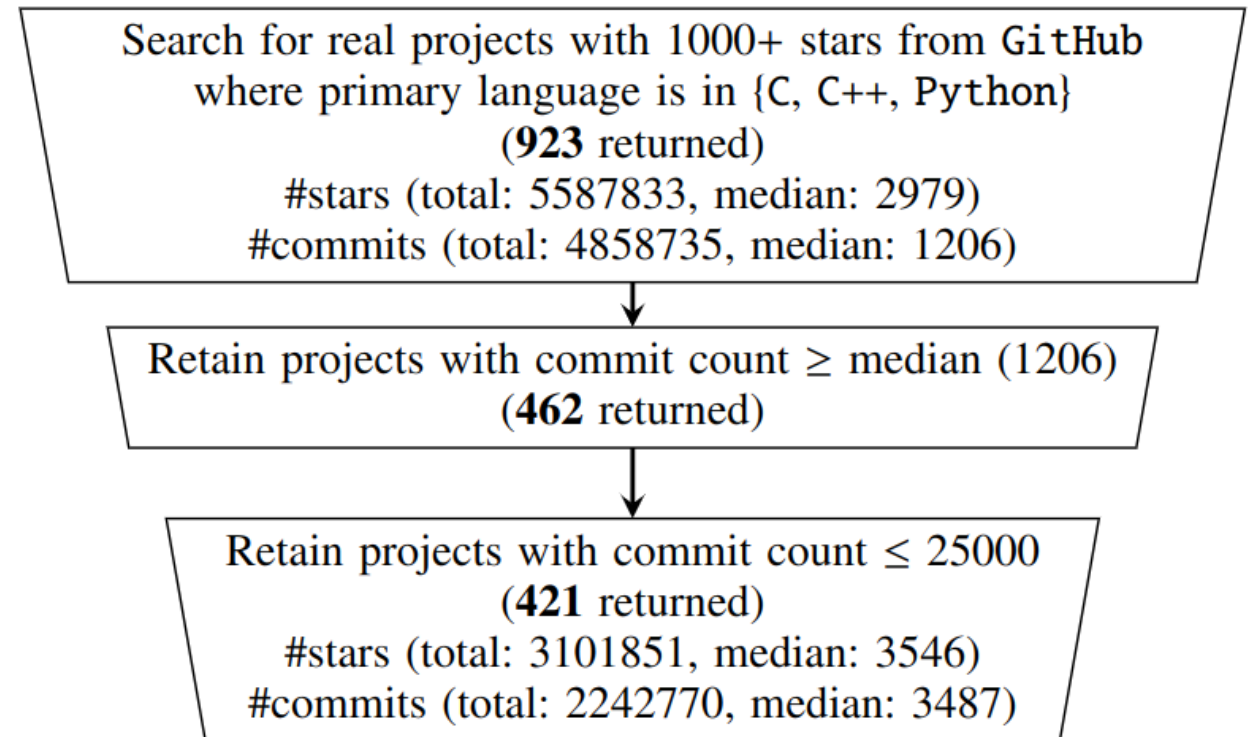


Fig. 2. Our project selection process.

Example workflow...

To demonstrate with examples, we chose a project named **Geocoder** which is written in **Python**

```
geocoder/komoot.py  git diff
@@ -22,7 +22,7 @@ def lng(self):
22 22     @property
23 23     def bbox(self):
24 24         extent = self.raw['properties'].get('extent')
25 -         if extent:
25 +         if extent and all(extent):
26 26             west = extent[0]
27 27             north = extent[1]
28 28             east = extent[2]
```



Geocoder: Simple, Consistent

Release v1.38.1. ([Installation](#))

Simple and consistent geocoding library written in Python.

Many online providers such as Google & Bing have geocoding services but do not include Python libraries and have different JSON responses between

It can be very difficult sometimes to parse a particular geocoding provider's response as each one of them have their own JSON schema.

Here is a typical example of retrieving a Lat & Lng from Google using the Geocoder library. It shouldn't be this hard.

```
>>> import requests
>>> url = 'https://maps.googleapis.com/maps/api/geocode/json?address=Mountain View, CA'
>>> params = {'sensor': 'false', 'address': 'Mountain View, CA'}
>>> r = requests.get(url, params=params)
>>> results = r.json()['results']
>>> location = results[0]['geometry']['location']
>>> location['lat'], location['lng']
(37.3860517, -122.0838511)
```

Support

If you are having issues we would love to hear from you. Just [hit me up](#). You can alternatively raise an issue [here](#) on Github.

Now lets use Geocoder to do the same task.

```
>>> import geocoder
>>> g = geocoder.google('Mountain View, CA')
>>> g.latlng
(37.3860517, -122.0838511)
```

Before

```
18 @property
19 def lng(self):
20     return self.raw['geometry']['coordinates'][0]
21
22 @property
23 def bbox(self):
24     extent = self.raw['properties'].get('extent')
25 -     if extent:
26         west = extent[0]
27         north = extent[1]
28         east = extent[2]
29         south = extent[3]
30     return BBox.factory([south, west, north, east]).as_dict
31
32 @property
33 def address(self):
34     # Ontario, Canada
35     address = ', '.join([self.state, self.country])
```

After

```
18 @property
19 def lng(self):
20     return self.raw['geometry']['coordinates'][0]
21
22 @property
23 def bbox(self):
24     extent = self.raw['properties'].get('extent')
25 +     if extent and all(extent):
26         west = extent[0]
27         north = extent[1]
28         east = extent[2]
29         south = extent[3]
30     return BBox.factory([south, west, north, east]).as_dict
31
32 @property
33 def address(self):
34     # Ontario, Canada
35     address = ', '.join([self.state, self.country])
```

Loc: [before: 25, after: 25]

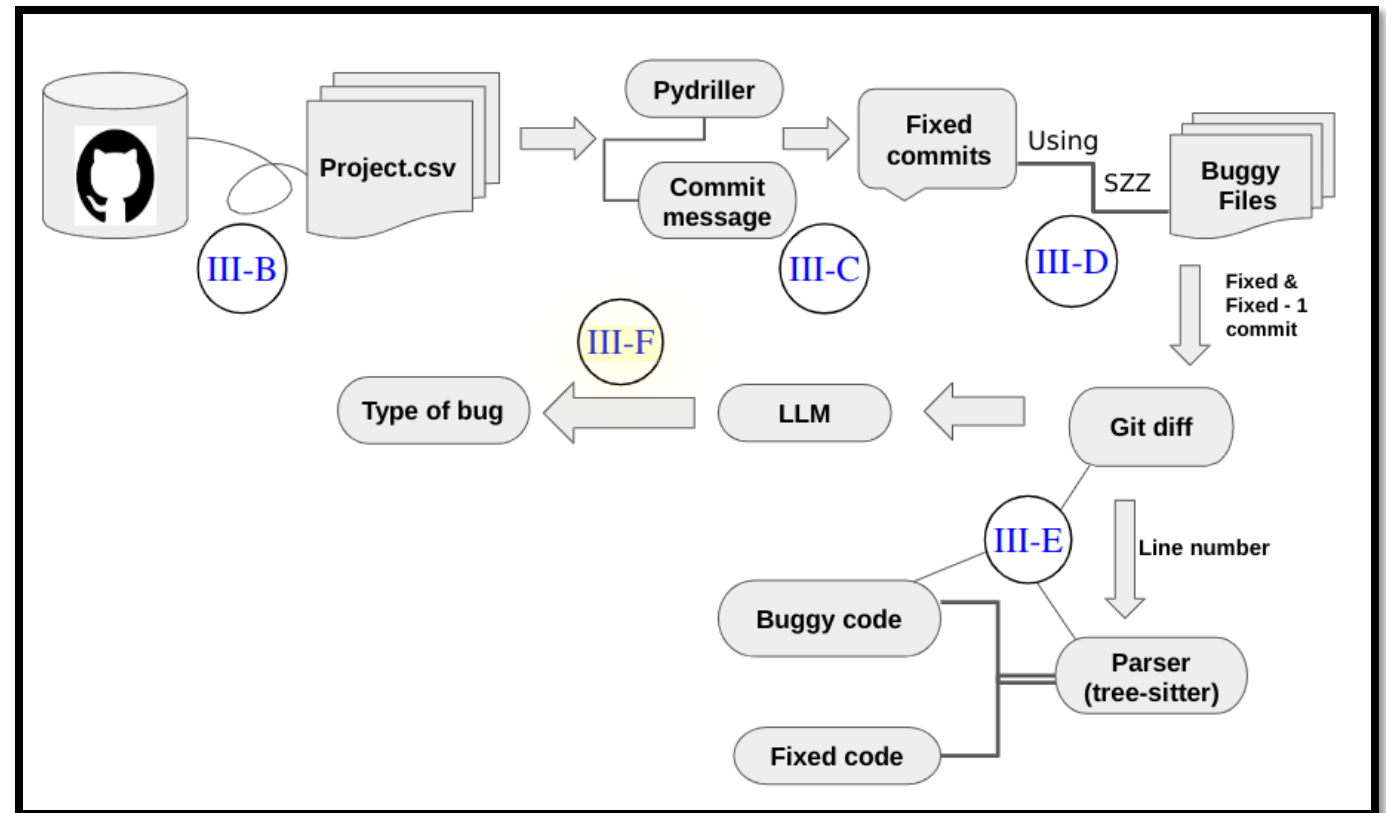
Bug Type: fix komoot.py when extent is none

Commit Message: fix calls to BBox with invalid input

Bug Fix Dataset Acquisition Flowchart with Precise Context

Data acquisition is done in following ways:

- Project Selection
- **Bug fix commits**
- Extracting Bug and fixed code Snippets
- Type of the bug



Collection of Bug-fix Commits

The collection of bug-fixing commits involved multiple strategies:

- Commit messages were analyzed to identify bug fix commits, leveraging keywords related to bug fixes. A total of **52 keywords** are used.
- Also, **Regular expressions** were utilized to perform precise searches for bug fix commit patterns
- The **SZZ algorithm** was applied to trace back and identify the buggy files associated with bug fix commits

“fixed ”, “ bug”, “fixes ”, “fix ”, “ fix”, “ fixed”, “ fixes”, “crash”, “solves”, “ resolves”, “resolves ”, “ issue”, “issue ”, “regression”, “fall back”, “assertion”, “coverity”, “reproducible”, “stack-wanted”, “steps-wanted”, “testcase”, “failur”, “fail”, “npe ”, “ npe”, “except”, “broken”, “differential testing”, “error”, “hang ”, “ hang”, “test fix”, “steps to reproduce”, “crash”, “assertion”, “failure”, “leak”, “stack trace”, “heap overflow”, “freez”, “problem ”, “ problem”, “ overflow”, “overflow ”, “avoid ”, “ avoid”, “workaround ”, “ workaround”, “break ”, “ break”, “ stop”, “stop ”

Commit messages with
52 keywords [6] (c.f. Sec. III-C)

```
+ RegEx. [7]: ‘ .((solv(ed|es|e|ing))  
| (fix(s|es|ing|ed)?))’  
| ((error|bug|issue)(s)?)) .’
```

→ SZZ Algorithm [8]

[J. Sliwinski, T. Zimmermann, and A. Zeller; (2005)] SZZ takes a commit (**bug-fixing**) as the input and returns a list of commits (**bug-introducing**) that last **added** the **deleted** lines in the input commit.

Example workflow...

To demonstrate with examples, we chose a project named **Geocoder** which is written in **Python**



Geocoder: Simple, Consistent

Release v1.38.1. (Installation)

Simple and consistent geocoding library written in Python.

Many online providers such as Google & Bing have geocoding services, these providers do not include Python libraries and have different JSON responses between each other. It can be very difficult sometimes to parse a particular geocoding provider since each one of them have their own JSON schema.

Here is a typical example of retrieving a Lat & Lng from Google using Python, things shouldn't be this hard.

```
>>> import requests
>>> url = 'https://maps.googleapis.com/maps/api/geocode/json'
>>> params = {'sensor': 'false', 'address': 'Mountain View, CA'}
>>> r = requests.get(url, params=params)
>>> results = r.json()['results']
>>> location = results[0]['geometry']['location']
>>> location['lat'], location['lng']
(37.3860517, -122.0838511)
```

Support

If you are having issues we would love to hear from you. Just hit me up. You can alternatively raise an issue here on Github.

```
>>> import geocoder
>>> g = geocoder.google('Mountain View, CA')
>>> g.latlng
(37.3860517, -122.0838511)
```

```
22 22 @property
23 23 def bbox(self):
24 24     extent = self.raw['properties'].get('extent')
25 -   if extent:
26 +   if extent and all(extent):
27     west = extent[0]
28     north = extent[1]
29     east = extent[2]
```

```
18 @property
19 def lng(self):
20     return self.raw['geometry']['coordinates'][0]
21
22 @property
23 def bbox(self):
24     extent = self.raw['properties'].get('extent')
25 -   if extent:
26     west = extent[0]
27     north = extent[1]
28     east = extent[2]
29     south = extent[3]
30     return BBox.factory([south, west, north,
31                          east]).as_dict
32
33 @property
34 def address(self):
35     # Ontario, Canada
36     address = ', '.join([self.state, self.country])
```

```
18 @property
19 def lng(self):
20     return self.raw['geometry']['coordinates'][0]
21
22 @property
23 def bbox(self):
24     extent = self.raw['properties'].get('extent')
25 +   if extent and all(extent):
26     west = extent[0]
27     north = extent[1]
28     east = extent[2]
29     south = extent[3]
30     return BBox.factory([south, west, north,
31                          east]).as_dict
32
33 @property
34 def address(self):
35     # Ontario, Canada
36     address = ', '.join([self.state, self.country])
```

Loc: [before: 25, after: 25]

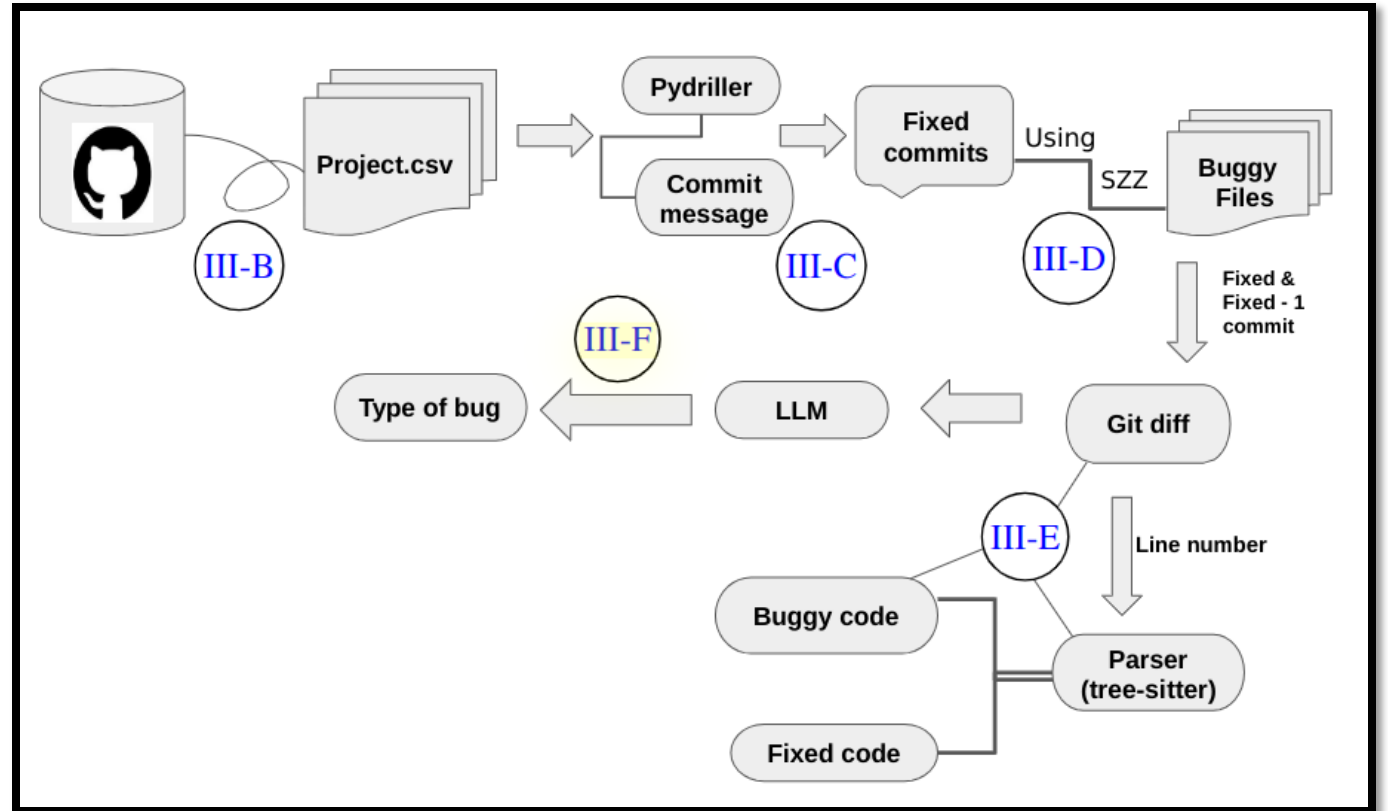
Bug Type: fix komoot.py when extent is none

Commit Message: fix calls to BBox with invalid input

Bug Fix Dataset Acquisition Flowchart with Precise Context

Data acquisition is done in following ways:

- Project Selection
- Bug fix commits
- **Extracting Bug and fixed code Snippets**
- Type of the bug

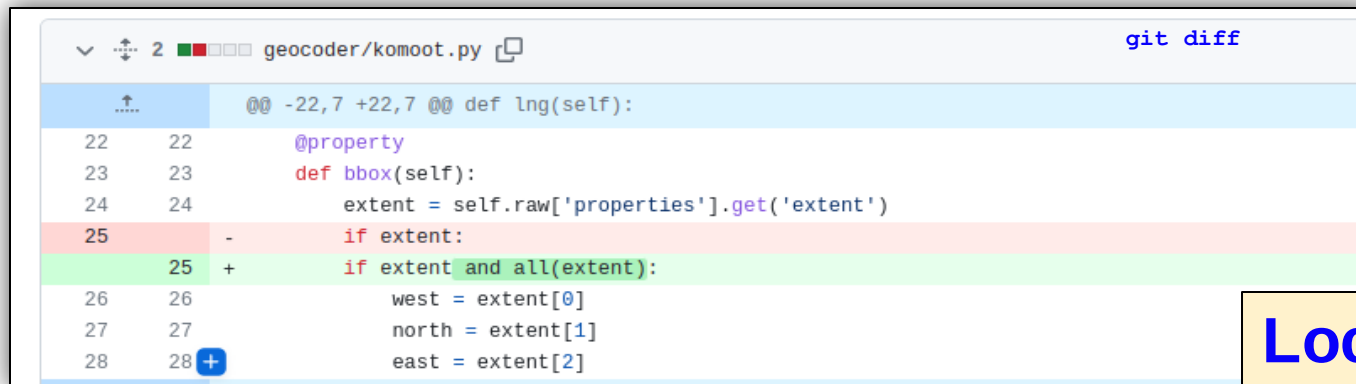


Extracting Code Snippets

The extraction of code snippets is done as follows:

Step 1:

- **Location:** The location of bug is found using `git diff` between `fixed - 1` and `fixed` commit. The location here indicates the **line number** in which the bug is detected and fixed.



```
git diff
@@ -22,7 +22,7 @@ def lng(self):
22 22         @property
23 23         def bbox(self):
24 24             extent = self.raw['properties'].get('extent')
25 -         if extent:
25 +         if extent and all(extent):
26 26             west = extent[0]
27 27             north = extent[1]
28 28             east = extent[2]
```

a **modification**
typically involves an
addition after a **deletion**

Loc: [before: 25, after: 25]

Bug Type: fix komoot.py when extent is none

Commit Message: fix calls to BBox with invalid input

Extracting Code Snippets

Step 2:

- **Buggy Code:** The location[before] obtained is used to extract the code snippet using **Tree-Sitter** parser.
- **Fixed Code:** The location[after] obtained is used to extract the code snippet using **Tree-Sitter** parser.

```
18     @property
19     def lng(self):
20         return self.raw['geometry']['coordinates'][0]
21
22     @property
23     def bbox(self):
24         extent = self.raw['properties'].get('extent')
25 -     if extent:
26         west = extent[0]
27         north = extent[1]
28         east = extent[2]
29         south = extent[3]
30         return BBox.factory([south, west, north,
31                             east]).as_dict
32
33     @property
34     def address(self):
35         # Ontario, Canada
36         address = ', '.join(
```

```
18     @property
19     def lng(self):
20         return self.raw['geometry']['coordinates'][0]
21
22     @property
23     def bbox(self):
24         extent = self.raw['properties'].get('extent')
25 +     if extent and all(extent):
26         west = extent[0]
27         north = extent[1]
28         east = extent[2]
29         south = extent[3]
30         return BBox.factory([south, west, north,
31                             east]).as_dict
32
33     @property
34     def address(self):
35         if state, self.country]]
```

Loc: [before: 25, after: 25]

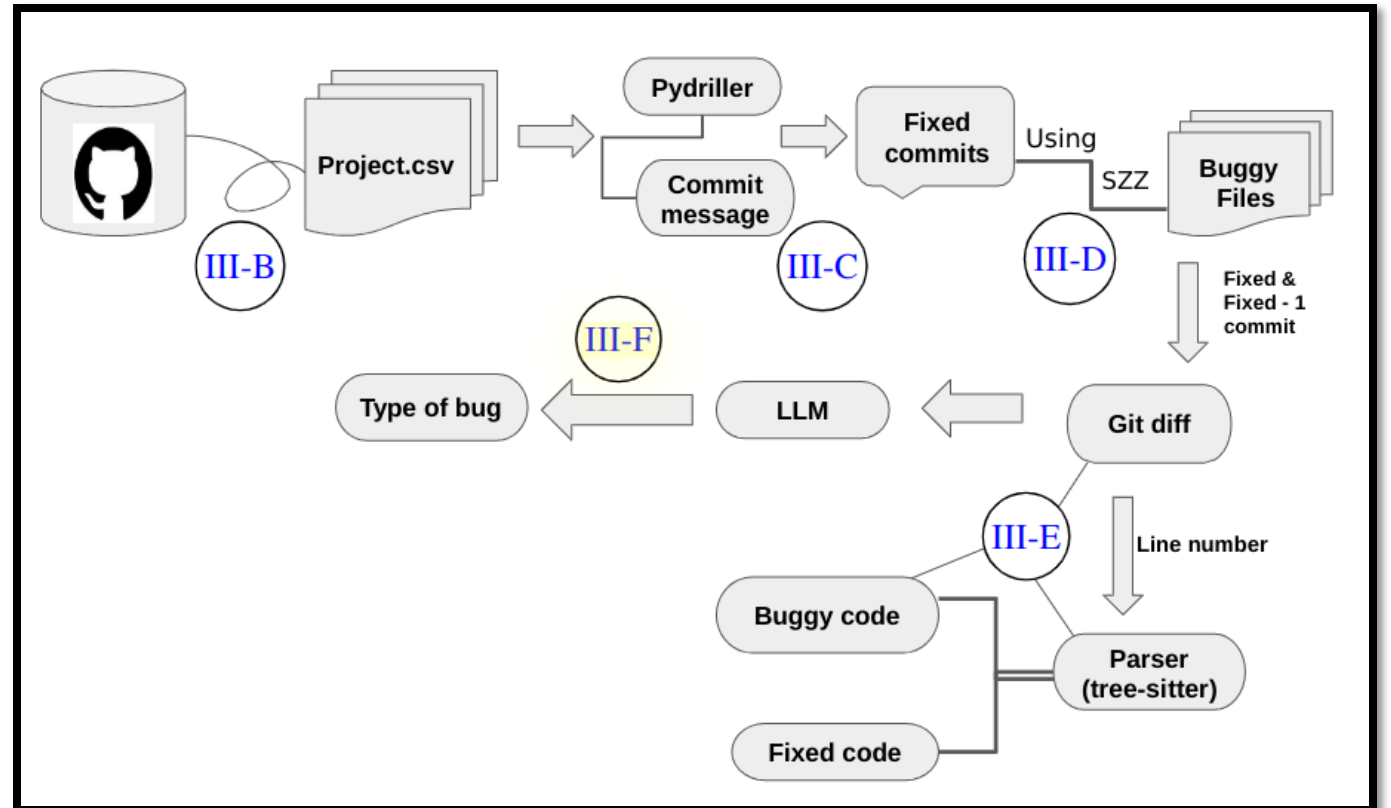
Bug Type: fix komoot.py when extent is none

Commit Message: fix calls to BBox with
invalid input

Bug Fix Dataset Acquisition Flowchart with Precise Context

Data acquisition is done in following ways:

- Project Selection
- Bug fix commits
- Extracting Bug and fixed code Snippets
- **Type of the bug**



Inferencing the type of Bug

The type of bug is inferenced using a **commit message generator** Large Language Model (LLM).

URL: <https://huggingface.co/mamiksik/CommitPredictorT5>

The LLM takes `git-diff` of fixed - 1 and fixed commits as input and generates the summary of diff which is nothing but a fix and thus inferred as **bug type**.

CommitPredictorT5

This model is a fine-tuned version of [Salesforce/codet5-base-multi-sum](#) on the None dataset. It achieves the following results on the evaluation set:

- Loss: 2.4669
- Bleu: 0.0002
- Precisions: [0.003189792663476874, 0.00016826518593303046, 0.000321853878339234, 0.0036900369003690036]
- Brevity Penalty: 0.2394
- Length Ratio: 0.4116
- Translation Length: 10658
- Reference Length: 25896

Hosted inference API

Text2Text Generation

Your sentence here...

Loc: [before: 25, after: 25]
Bug Type: fix komoot.py when extent is none
Commit Message: fix calls to BBox with invalid input

JAVA Example

Before Bug fix:

```
31 * Functional tests for the {@link
    AutoFactoryProcessor}.
32 */
33 public class AutoFactoryProcessorTest {
34     @Test public void simpleClass() {
35         ASSERT.about(javaSource())
36         .that(...("tests/SimpleClass.java"))
37         .processedWith(new AutoFactoryProcessor())
38         .compilesWithoutError()
39         .and().generatesSources(JavaFileObjects
40         ...("tests/SimpleClassFactory.java"));
41     }
42
43     @Test public void publicClass() {
```

After Bug fix:

```
30 * Functional tests for the {@link
    AutoFactoryProcessor}.
31 */
32 public class AutoFactoryProcessorTest {
33     @Test public void simpleClass() {
34         ASSERT.about(javaSource())
35         .that(...("good/SimpleClass.java"))
36         .processedWith(new AutoFactoryProcessor())
37         .compilesWithoutError()
38         .and().generatesSources(JavaFileObjects...
39         ("expected/SimpleClassFactory.java"));
40     }
41
42     @Test public void publicClass() {
```

Loc: [before: 36, after: 35]

Bug Type: fix coverage for autofactoryprocessor tests

Commit Message: Break up the test files so that it's clear which test files are expected to work.

C++ Example

```
287 } Before Bug Fix:
288
289
290 static void gc_allocation_map_remove(...)
291 {
292     // ignores unknown keys
293     size_t index = gc_hash(ptr) % am->capacity;
294     Allocation* cur = am->allocs[index];
295     Allocation* prev = NULL;
296     while(cur != NULL) {
297         if (cur->ptr == ptr) {
298             // found it
299             if (!prev) {
300                 // first item in list
301                 am->allocs[index] = cur->next;
302             } else {
303                 // not the first item in the list
304                 prev->next = cur->next;
305             }
306             gc_allocation_delete(cur);
307             am->size--;
308         } else {
309             // move on
310             prev = cur;
311         }
312         cur = cur->next;
313     }
314     if (allow_resize) {
315         gc_allocation_m
316     }
317 }
```

```
287 } After Bug Fix:
288
289 }
290
291 static void gc_allocation_map_remove(...)
292 {
293     // ignores unknown keys
294     size_t index = gc_hash(ptr) % am->capacity;
295     Allocation* cur = am->allocs[index];
296     Allocation* prev = NULL;
297     while(cur != NULL) {
298         Allocation* cur_next = cur->next;
299         if (cur->ptr == ptr) {
300             // found it
301             if (!prev) {
302                 // first item in list
303                 am->allocs[index] = cur->next;
304             } else {
305                 // not the first item in the list
306                 prev->next = cur->next;
307             }
308             gc_allocation_delete(cur);
309             am->size--;
310         } else {
311             // move on
312             prev = cur;
313         }
314         cur = cur_next;
315     }
316     if (allow_resize) {
317 }
```

Loc: [before: 314, after: 315]

Bug Type: use cur_next instead of cur->next in gc_allocation_map_remove

Commit Message: Crash fixing.

Context of a bug

- specific **circumstances** or **conditions** in which a bug **occurs** or **manifests** itself.
- Conditions are typically represented by enclosing code constructs in the source language
- sequence of statements leading to the bug, and the dependencies between different code modules or functions

Within function

Here the **context** of bug is the function in which the bug is present and three lines below and above the function

```
git diff
geocoder/komoot.py
@@ -22,7 +22,7 @@ def lng(self):
22 22 @property
23 23 def bbox(self):
24 24     extent = self.raw['properties'].get('extent')
25 -     if extent:
25 +     if extent and all(extent):
26 26         west = extent[0]
27 27         north = extent[1]
28 28         east = extent[2]
```



Geocoder: Simple, Consistent

Release v1.38.1. ([Installation](#))

Simple and consistent geocoding library written in Python.

Many online providers such as Google & Bing have geocoding services but do not include Python libraries and have different JSON responses between

It can be very difficult sometimes to parse a particular geocoding provider as one of them have their own JSON schema.

Here is a typical example of retrieving a Lat & Lng from Google using geocoder which shouldn't be this hard.

```
>>> import requests
>>> url = 'https://maps.googleapis.com/maps/api/geocode/json?address=Mountain View, CA'
>>> params = {'sensor': 'false', 'address': 'Mountain View, CA'}
>>> r = requests.get(url, params=params)
>>> results = r.json()['results']
>>> location = results[0]['geometry']['location']
>>> location['lat'], location['lng']
(37.3860517, -122.0838511)
```

Support

If you are having issues we would love to hear from you. Just [hit me up](#). You can alternatively raise an issue [here](#) on Github.

Now lets use Geocoder to do the same task.

```
>>> import geocoder
>>> g = geocoder.google('Mountain View, CA')
>>> g.latlng
(37.3860517, -122.0838511)
```

Before

```
18 @property
19 def lng(self):
20     return self.raw['geometry']['coordinates'][0]
21
22 @property
23 def bbox(self):
24     extent = self.raw['properties'].get('extent')
25 -     if extent:
26         west = extent[0]
27         north = extent[1]
28         east = extent[2]
29         south = extent[3]
30     return BBox.factory([south, west, north, east]).as_dict
31
32 @property
33 def address(self):
34     # Ontario, Canada
35     address = ', '.join([self.state, self.country])
```

After

```
18 @property
19 def lng(self):
20     return self.raw['geometry']['coordinates'][0]
21
22 @property
23 def bbox(self):
24     extent = self.raw['properties'].get('extent')
25 +     if extent and all(extent):
26         west = extent[0]
27         north = extent[1]
28         east = extent[2]
29         south = extent[3]
30     return BBox.factory([south, west, north, east]).as_dict
31
32 @property
33 def address(self):
34     # Ontario, Canada
35     address = ', '.join([self.state, self.country])
```

Loc: [before: 25, after: 25]

Bug Type: fix komoot.py when extent is none

Commit Message: fix calls to BBox with invalid input

Outside function

The **context** of the bug here is five lines above and below it.



Geocoder: Simple, Consistent

Release v1.38.1. ([Installation](#))

Simple and consistent geocoding library written in Python.

Many online providers such as Google & Bing have geocoding services, these providers do not include Python libraries and have different JSON responses between each other.

It can be very difficult sometimes to parse a particular geocoding provider since each one of them have their own JSON schema.

Here is a typical example of retrieving a Lat & Lng from Google using Python, this shouldn't be this hard.

Geocoder is a simple and consistent geocoding library written in Python. Dealing with multiple different geocoding providers such as Google, Bing, OSM & many more has never been easier.

Support

If you are having issues we would love to hear from you. Just [hit me up](#). You can alternatively raise an issue [here](#) on Github.

```
>>> import requests
>>> url = 'https://maps.googleapis.com/maps/api/geocode/json'
>>> params = {'sensor': 'false', 'address': 'Mountain View, CA'}
>>> r = requests.get(url, params=params)
>>> results = r.json()['results']
>>> location = results[0]['geometry']['location']
>>> location['lat'], location['lng']
(37.3860517, -122.0838511)
```

Now let's use Geocoder to do the same task.

```
>>> import geocoder
>>> g = geocoder.google('Mountain View, CA')
>>> g.latlng
(37.3860517, -122.0838511)
```

```
git diff
@@ -7,7 +7,7 @@ class Bing(Base):
7       #http://msdn.microsoft.com/en-us/library/ff701713.aspx
8       name = 'Bing'
9       url = 'http://dev.virtualearth.net/REST/v1/Locations'
10      -   key = ''
10      +   key = 'AtnSnX1rEHR3yTUGC3EHkD6Qi3NNB-PABa_F9F8zvLxxvt8A7aYdiG3bGM_PorOq'
11
12      def __init__(self, location, key=''):
13          self.location = location
```

```
Before
4
5
6      class Bing(Base):
7          #http://msdn.microsoft.com/en-
8          us/library/ff701713.aspx
9          name = 'Bing'
10         url =
11             'http://dev.virtualearth.net/REST/v1/Locations'
12         -   key = ''
13
14     def __init__(self, location, key=''):
15         self.location = location
16
17     if not key:
18         key = self.key
```

```
After
4
5
6      class Bing(Base):
7          #http://msdn.microsoft.com/en-
8          us/library/ff701713.aspx
9          name = 'Bing'
10         url =
11             'http://dev.virtualearth.net/REST/v1/Locations'
12         +   key = 'AtnSnX1rEHR3yTUGC3EHkD6Qi3NNB-
13             PABa_F9F8zvLxxvt8A7aYdiG3bGM_PorOq'
14
15     def __init__(self, location, key=''):
16         self.location = location
17
18     if not key:
19         key = self.key
```

Loc: [before: 10, after: 10]

Bug Type: add key to bing

Commit Message: Fixed bbox & Added keys

What is Precise Code Context ?

```
18     @property
19     def lng(self):
20         return self.raw['geometry']['coordinates'][0]
21
22     @property
23     def bbox(self):
24         extent = self.raw['properties'].get('extent')
25 -     if extent:
26         west = extent[0]
27         north = extent[1]
28         east = extent[2]
29         south = extent[3]
30         return BBox.factory([south, west, north,
31                             east]).as_dict
32
33     @property
34     def address(self):
35         # Ontario, Canada
36         address = ', '.join([self.state, self.country])
```

```
18     @property
19     def lng(self):
20         return self.raw['geometry']['coordinates'][0]
21
22     @property
23     def bbox(self):
24         extent = self.raw['properties'].get('extent')
25 +     if extent and all(extent):
26         west = extent[0]
27         north = extent[1]
28         east = extent[2]
29         south = extent[3]
30         return BBox.factory([south, west, north,
31                             east]).as_dict
32
33     @property
34     def address(self):
35         # Ontario, Canada
36         address = ', '.join([self.state, self.country])
```

Loc: [before: 25, after: 25]

Bug Type: fix komoot.py when extent is none

Commit Message: fix calls to BBox with invalid input

Combining everything about
bug makes it **precise code
context**

Minecraft: Automated Mining of Software Bug Fixes with Precise Code Context

- Automatic **language-agnostic** algorithm (C/C++, Java and Python)
- Extract code snippets before and after bug fix
- **Precise location** of bugs (file, function and statement (line no.))
- Bug type
- A **dataset** of bug-fix pairs

COMPARISON OF MINECRAFT WITH EXISTING REPOSITORY MINED DATASETS			
	<i>FixJs</i> [3]	<i>Tufano et al.</i> [4]	<i>Minecraft</i> (this paper)
<i>Introduced in</i>	MSR 2022	TOSEM 2019	This paper
<i>Language</i>	JavaScript	Java	C, C++, Python, and Java
<i>Granularity</i>	Function	Function	File, Function, and Statement (ranges)
<i>#Commits</i>	~2M	~787K	~2.2M
<i>#Bug-Fixes</i>	300K	~2.3M	~3.29M
<i>Dataset size</i>	5.47GB	~7GB	28.8GB
<i>Bug Detection Strategy</i>	Commit messages with 6 keywords [5]: ["fix", "solve", "bug", "issue", "problem", "error"]	Commit messages with 6 keywords [5]: ["fix", "solve", "bug", "issue", "problem", "error"]	Commit messages with 52 keywords [6] (c.f. Sec. III-C) + RegEx. [7]: <code>'((solv(ed es e ing)) (fix(s es ing ed)?))' ((error bug issue)(s)?))'.</code> + SZZ Algorithm [8]

Dataset

- Has **five** columns
- Size of dataset: **28.8GB**
- A total of **~3.29M** bug-fix pairs (C/C++, Python, Java)
 - **421** projects

Col#	Name	Type	Description
1	Before Bug fix	String	Code lines (+ context) before bug fix
2	After Bug fix	String	Code lines (+ context) after bug fix
3	Location	List of ints.	Line #s of the bug and it's fix
4	Bug type	String	Bug type inferred from LLM using git dif
5	Commit Message	String	Code review and version control



<https://zenodo.org/records/8164641>

Minecraft

*"Concrete workflow. The proposed workflow can **automatically** mine the **bug fixes** from GitHub repositories, with specific information that basically meets the expectations"*

-Reviewer (ASE)

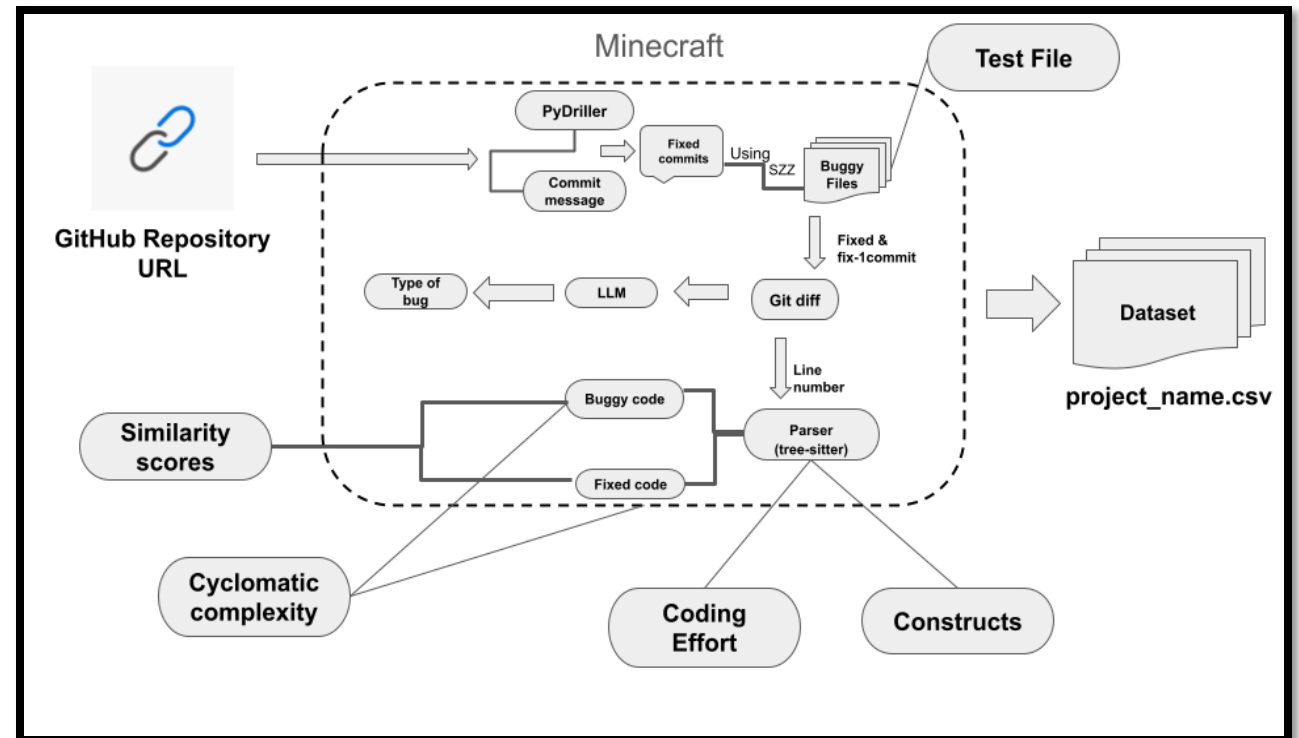
MineCPP: Mining Bug Fix Pairs and Their Structures

Published in FSE 2024 (CORE A*) Demonstrations track

<https://2024.esec-fse.org/track/fse-2024-demonstrations>

MineCPP: An extension of Minecraft

- Minecraft overcame limitations of existing datasets
- Is the data produced useful enough for comprehensive bug analysis?
- **MineCPP (Minecraft ++)** - extend the capabilities of bug mining methodology (Minecraft) by introducing **innovative features** and **metrics** not present in existing tools.



MineCPP: Additional Features

- **Coding Effort**
- **Test Case indicator**
- **Constructs surrounding the bug**
- **Cyclomatic Complexity**
- **Similarity Score**

MineCPP: Additional Features

- **Coding Effort**
- Test Case indicator
- Constructs surrounding the bug
- Cyclomatic Complexity
- Similarity Score

Coding Effort

- **Novel metric** introduced in MineCPP
- The **#nodes** traversed before reaching the **buggy node** of an Abstract Syntax Tree (**AST**)
- Quantifies the complexity and effort required by author/developer to introduce bugs within source code.

Example: Coding Effort

- Buggy code for **addition**
- **AST** of Buggy Code
- **Buggy node**
- AST nodes Traversal till buggy node
- Coding Effort: **24**

```
1 import sys
2
3 def add(a: int, b: int):
4     return a - b
5
6 num1 = int(input())
7 num2 = int(input())
8 print(add(num1, num2))
```

Buggy Code

Module -> import_statement -> import -> dotted_name -> identifier->
function_definition -> def -> identifier -> parameters -> (-> typed_parameter ->
identifier -> : -> type -> identifier -> , -> typed_parameter -> identifier -> : -> type->
identifier ->) -> : -> block

```
module (0, 4)
  import_statement (0, 0)
    import (0, 0)
      dotted_name (0, 0)
        identifier (0, 0)
  function_definition (2, 3)
    def (2, 2)
      identifier (2, 2)
      parameters (2, 2)
        ( (2, 2)
          typed_parameter (2, 2)
            identifier (2, 2)
            : (2, 2)
            type (2, 2)
            identifier (2, 2)
          , (2, 2)
          typed_parameter (2, 2)
            identifier (2, 2)
            : (2, 2)
            type (2, 2)
            identifier (2, 2)
        ) (2, 2)
      : (2, 2)
      block (3, 3)
        return_statement (3, 3)
          return (3, 3)
          binary_operator (3, 3)
            identifier (3, 3)
            - (3, 3)
            identifier (3, 3)
```

AST

MineCPP: Additional Features

- Coding Effort
- **Test Case indicator**
- Constructs surrounding the bug
- Cyclomatic Complexity
- Similarity Score

Test Case Indicator

- Indicates whether a bug-fix pair has a test case. **1** indicates presence and **0** indicates absence
- Navigates through the repository files to match regular expressions against files to find a test case.

```
re.compile(r'test[^\a-zA-Z]*{}|{}[^\a-zA-Z]*test'  
  
.format(re.escape(base_fileName_withoutExtension),  
  
re.escape(base_fileName_withoutExtension)), re.IGNORECASE)
```

Regular Expression

MineCPP: Additional Features

- Coding Effort
- Test Case indicator
- **Constructs surrounding the bug**
- Cyclomatic Complexity
- Similarity Score

Constructs surrounding the bug

- The “**Constructs**” metric **focuses** language constructs surrounding the bug context
- The metric considers the **frequency** of these constructs within the immediate **context** of the bug.
- Provides targeted insights into the nature and characteristics of bugs within software codebases.

Example: Constructs

- Buggy code
- **Context** of Buggy Code
- AST of bug and its context
- **Constructs**

```
1 import sys
2
3 def add(a: int, b: int):
4     return a - b
5
6 num1 = int(input())
7 num2 = int(input())
```

Buggy Code (context)

```
{'module': 1, 'import_statement': 1, 'import': 1, 'dotted_name': 1, 'identifier': 14,
'function_definition': 1, 'def': 1, 'parameters': 1, '(': 5, 'typed_parameter': 2, ':': 3, 'type':
2, ',': 1, ')': 5, 'block': 1, 'return_statement': 1, 'return': 1, 'binary_operator': 1, '-': 1,
'expression_statement': 1, 'assignment': 2, '=': 2, 'call': 4, 'argument_list': 4}
```

```
module (0, 7)
  import_statement (0, 0)
    import (0, 0)
      dotted_name (0, 0)
        identifier (0, 0)
      function_definition (2, 3)
        def (2, 2)
          identifier (2, 2)
          parameters (2, 2)
            ( (2, 2)
              typed_parameter (2, 2)
                identifier (2, 2)
                : (2, 2)
                type (2, 2)
                identifier (2, 2)
            ) (2, 2)
          , (2, 2)
          typed_parameter (2, 2)
            identifier (2, 2)
            : (2, 2)
            type (2, 2)
            identifier (2, 2)
          ) (2, 2)
        : (2, 2)
        block (3, 3)
          return_statement (3, 3)
            return (3, 3)
            binary_operator (3, 3)
              identifier (3, 3)
              - (3, 3)
              identifier (3, 3)
            expression_statement (5, 5)
              assignment (5, 5)
                identifier (5, 5)
                = (5, 5)
                call (5, 5)
                  identifier (5, 5)
                  argument_list (5, 5)
                    ( (5, 5)
                      call (5, 5)
                        identifier (5, 5)
                        argument_list (5, 5)
                          ( (5, 5)
                            ( (5, 5)
                              ) (5, 5)
                            ) (5, 5)
                          ) (5, 5)
                    ) (5, 5)
                  ) (5, 5)
              expression_statement (6, 6)
                assignment (6, 6)
                  identifier (6, 6)
                  = (6, 6)
                  call (6, 6)
                    identifier (6, 6)
                    argument_list (6, 6)
                      ( (6, 6)
                        call (6, 6)
                          identifier (6, 6)
                          argument_list (6, 6)
                            ( (6, 6)
                              ( (6, 6)
                                ) (6, 6)
                              ) (6, 6)
                            ) (6, 6)
                        ) (6, 6)
                      ) (6, 6)
                    ) (6, 6)
                ) (6, 6)
            ) (6, 6)
          ) (6, 6)
        ) (6, 6)
      ) (6, 6)
    ) (6, 6)
  ) (6, 6)
```

AST

MineCPP: Additional Features

- Coding Effort
- Test Case indicator
- Constructs surrounding the bug
- **Cyclomatic Complexity**
- Similarity Score

Cyclomatic Complexity

- The structural complexity of the code - the **#independent paths** through the source code
- Determines the **stability** and **maintainability** of the codebase following the resolution of the identified issue
- **Lizard** library
- Cyclomatic Complexity
(before and after): **1**

```
1 import sys
2
3 def add(a: int, b: int):
4     return a - b
5
6 num1 = int(input())
7 num2 = int(input())
8 print(add(num1, num2))
```

Buggy Code

```
1 import sys
2
3 def add(a: int, b: int):
4     return a + b
5
6 num1 = int(input())
7 num2 = int(input())
8 print(add(num1, num2))
```

Bug-fix Code

MineCPP: Additional Features

- Coding Effort
- Test Case indicator
- Constructs surrounding the bug
- Cyclomatic Complexity
- **Similarity Score**

Similarity Score

- Provides a percentage of **code similarity** between the buggy and fixed code snippets
- **BLEU, CrystalBLEU, and Code BERT Score** used to calculate code similarity

- BLEU – **0.924**
- CrystalBLEU – **0.935**
- Code BERT Score – **0.99**

```
1 import sys
2
3 def add(a: int, b: int):
4     return a - b
5
6 num1 = int(input())
7 num2 = int(input())
8 print(add(num1, num2))
```

Buggy Code

```
1 import sys
2
3 def add(a: int, b: int):
4     return a + b
5
6 num1 = int(input())
7 num2 = int(input())
8 print(add(num1, num2))
```

Bug-fix Code

MineCPP (tool): Installation and Usage

- MineCPP is a **Python** installable package and can be installed using

```
pip install minecpp
```

Usage:

Optional arguments: minecpp [-h] [-version] [-u U]

Argument	Description
-h, -help	Shows tool's description and usage
-version	Shows tool's version
-u U	Provide the GitHub repo link to analyze

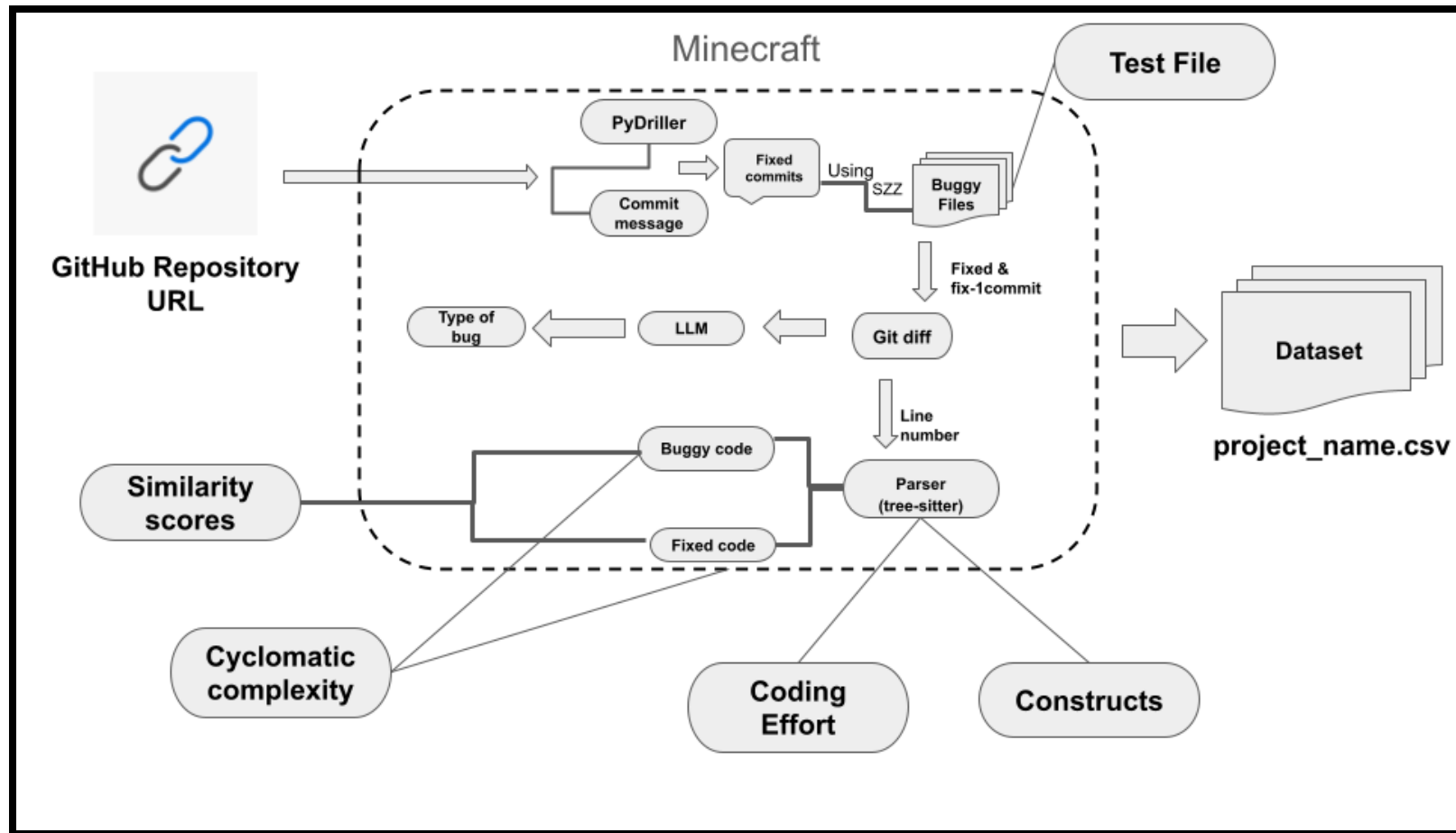
```
minecpp -u https://github.com/DenisCarriere/geocoder
```

<https://pypi.org/project/Minecpp>

Running the tool

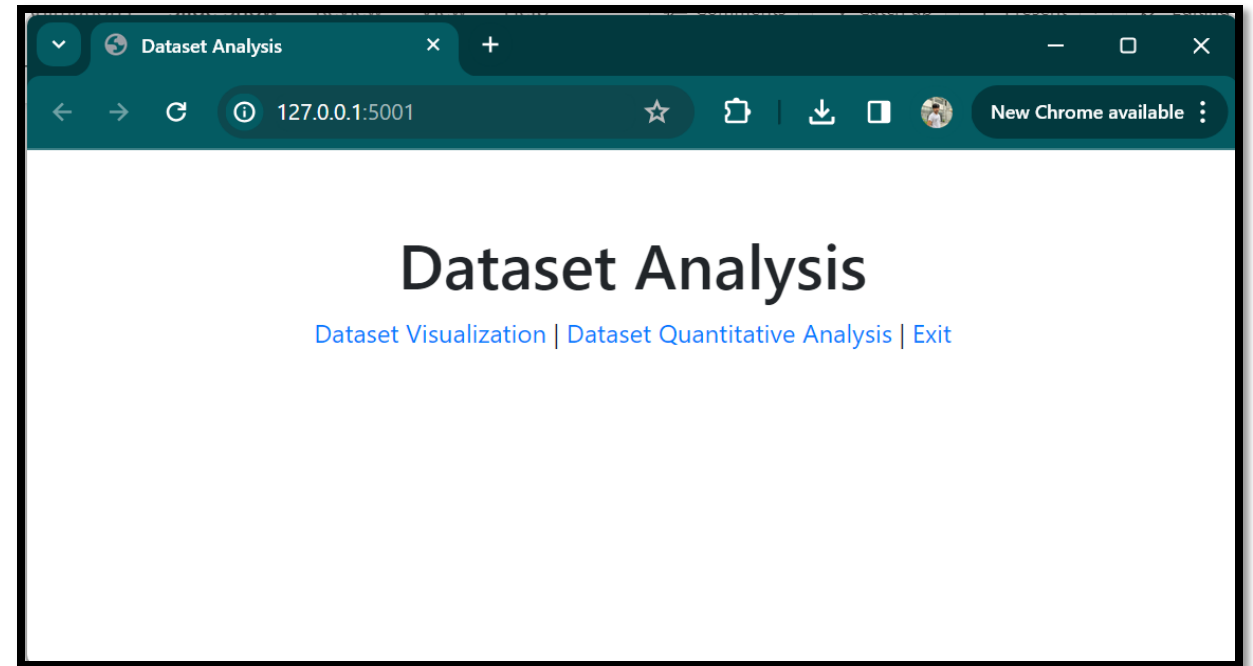
```
(minecpp) avulasaikrishna@hamilton:~/Sai_Work$ minecpp -u https://github.com/DenisCarriere/geocoder
[nltk_data] Downloading package punkt to
[nltk_data] /home/avulasaikrishna/nltk_data...
[nltk_data] Package punkt is already up-to-date!
Processing for project: geocoder
Getting fixed commits...
Repo: /home/avulasaikrishna/miniconda3/envs/minecpp/lib/python3.9/site-packages/Minecpp/geocoder/geocoder
Total comm: Corresponding commit msg: #108 Fixed Encoding with Google/Bing/ArcGIS
Processing Processing Commits: 18% | 48/261 [05:11<39:55, 11.25s/it]Processing for commit: 9d1ce989b4c
Processing 02ac8162329369491ad64971fa5b1
0a89510b8h Corresponding commit msg: #107 Fix URI duplicate in Kwargs
Corresponding commit msg: Import fix 52d8463a634
Processing Commits: 99% | 259/261 [27:59<00:12, 6.35s/it]Processing for commit: 48bc43a6dc4
761eb7761b5b0affe2424a8ca6c84
Corresponding commit msg: Merge pull request #342 from ThmsLa/master 5e4860c876c
Processing Commits: 100% | 260/261 [27:59<00:04, 4.52s/it]Processing for commit: 39b9999ec70
e61da9fa52fe9fe82a261ad70fa8b
Corresponding commit msg: Merge pull request #374 from vikaskyadav/patch-1 6833b13e355
Processing Commits: 100% | 261/261 [27:59<00:00, 6.46s/it]
Removing project folder: geocoder
* Serving Flask app 'Minecpp.app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit 533b8d7b24c
```

MineCPP: Flowchart



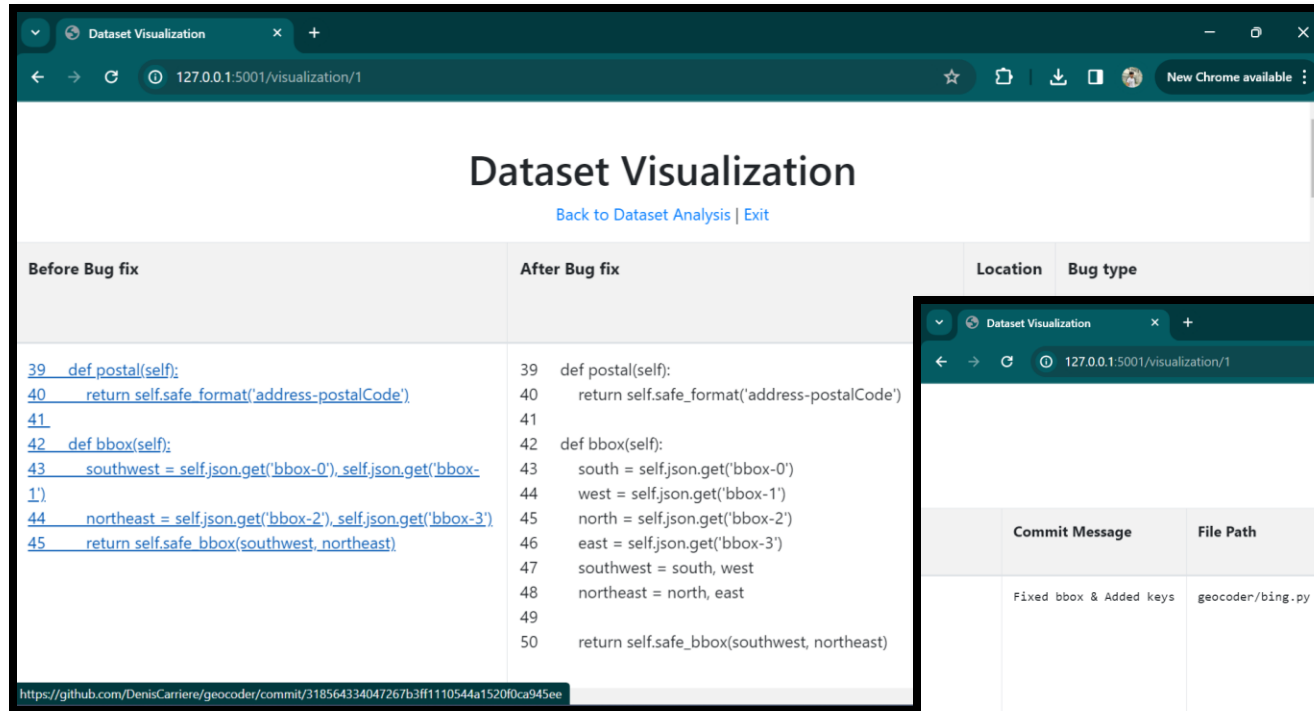
MineCPP: GUI (<https://youtu.be/In99irvbADE>)

- Dataset Visualization
 - Dataset of bug-fix pairs is displayed on webpage
 - **Interactive** webpage for bug-fix pairs of a repository
- Quantitative analysis
 - **Coding Effort vs Bug-fix pairs**
 - **Similarity Score vs Bug-fix pairs**



Dataset Visualization

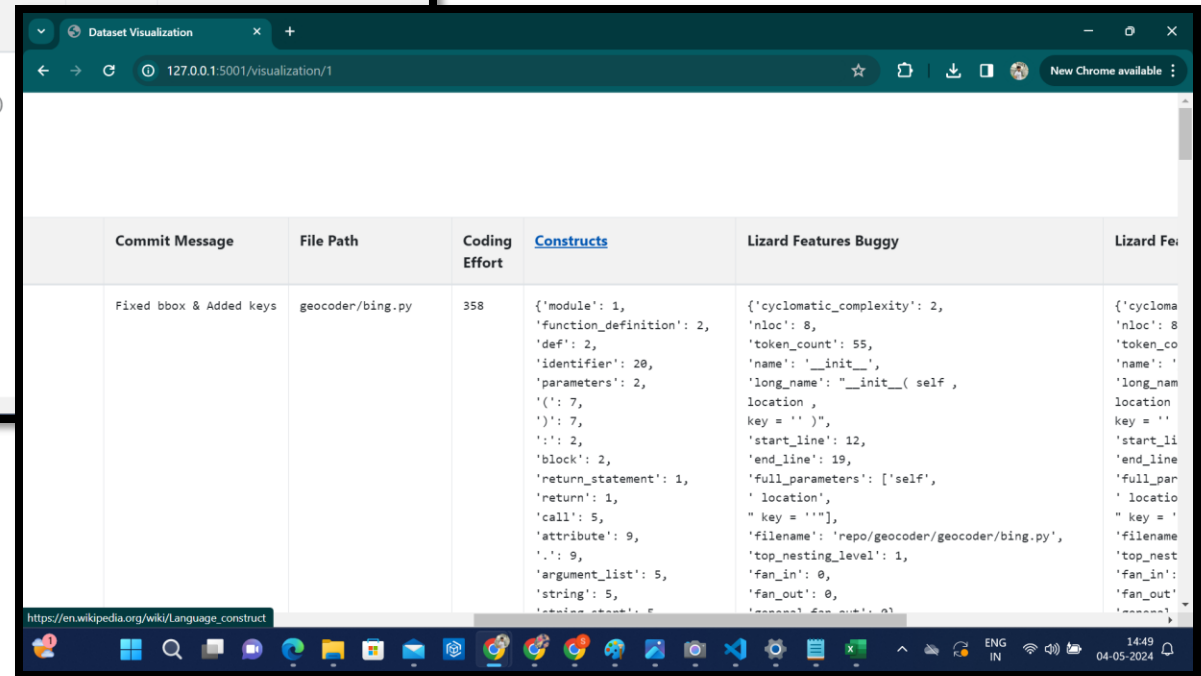
Clickable code snippets **redirects** to
GitHub source code



Before Bug fix	After Bug fix	Location	Bug type
<pre>39 def postal(self): 40 return self.safe_format('address-postalCode') 41 42 def bbox(self): 43 southwest = self.json.get('bbox-0'), self.json.get('bbox-1') 44 northeast = self.json.get('bbox-2'), self.json.get('bbox-3') 45 return self.safe_bbox(southwest, northeast)</pre>	<pre>39 def postal(self): 40 return self.safe_format('address-postalCode') 41 42 def bbox(self): 43 south = self.json.get('bbox-0') 44 west = self.json.get('bbox-1') 45 north = self.json.get('bbox-2') 46 east = self.json.get('bbox-3') 47 southwest = south, west 48 northeast = north, east 49 50 return self.safe_bbox(southwest, northeast)</pre>		

<https://github.com/DenisCarriere/geocoder/commit/318564334047267b3ff1110544a1520f0ca945ee>

Clickable columns
redirects to **Wikipedia**

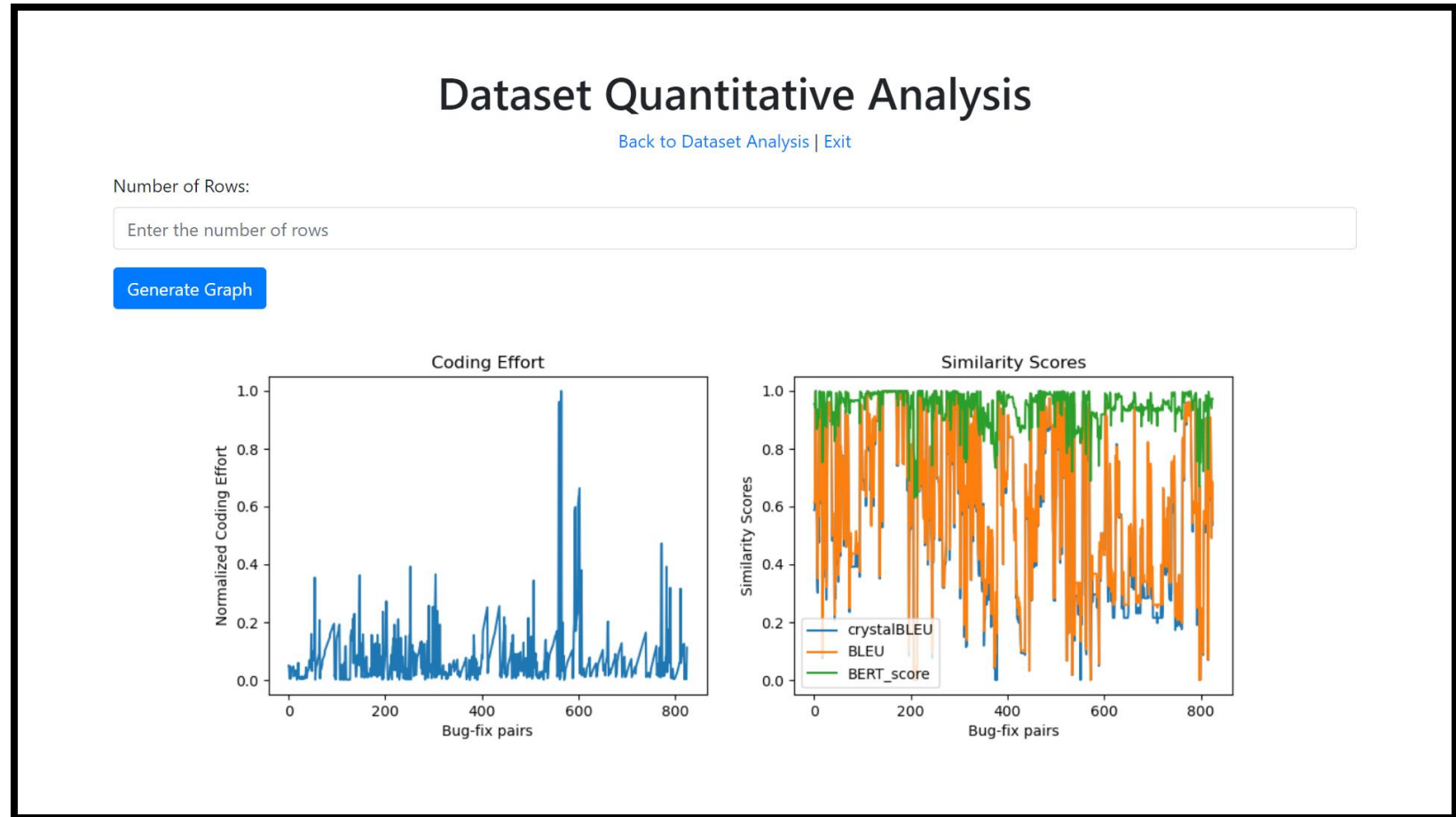


Commit Message	File Path	Coding Effort	Constructs	Lizard Features Buggy	Lizard Features
Fixed bbox & Added keys	geocoder/bing.py	358	<pre>{'module': 1, 'function_definition': 2, 'def': 2, 'identifier': 20, 'parameters': 2, '('': 7, ')': 7, ':': 2, 'block': 2, 'return_statement': 1, 'return': 1, 'call': 5, 'attribute': 9, ':': 9, 'argument_list': 5, 'string': 5, 'keyword': 5}</pre>	<pre>{'cyclomatic_complexity': 2, 'nloc': 8, 'token_count': 55, 'name': '__init__', 'long_name': '__init__(self , location , key = '')', 'start_line': 12, 'end_line': 19, 'full_parameters': ['self', 'location', 'key = ''], 'filename': 'repo/geocoder/geocoder/bing.py', 'top_nesting_level': 1, 'fan_in': 0, 'fan_out': 0, 'fan_out': 0}</pre>	<pre>{'cyclomatic_complexity': 2, 'nloc': 8, 'token_count': 55, 'name': '__init__', 'long_name': '__init__(self , location , key = '')', 'start_line': 12, 'end_line': 19, 'full_parameters': ['self', 'location', 'key = ''], 'filename': 'repo/geocoder/geocoder/bing.py', 'top_nesting_level': 1, 'fan_in': 0, 'fan_out': 0, 'fan_out': 0}</pre>

https://en.wikipedia.org/wiki/Language_construct

Dataset Quantitative Analysis

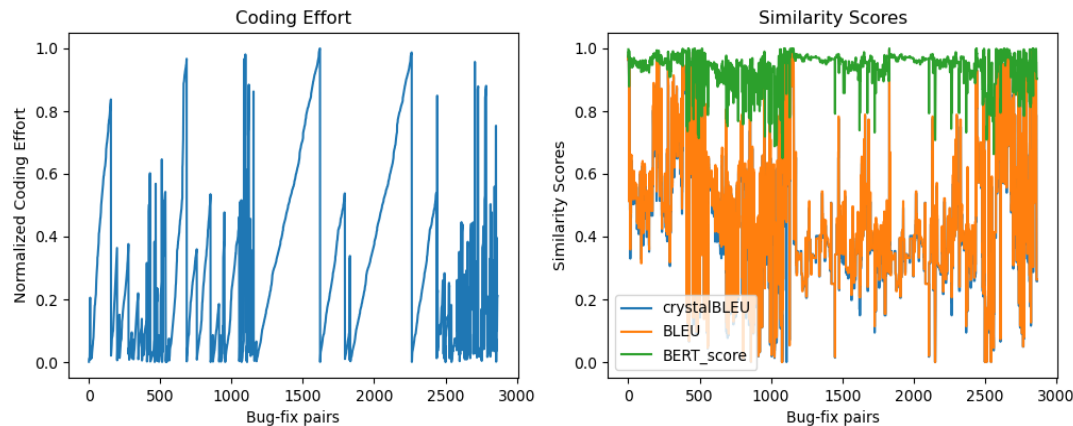
Example for Geocoder repository



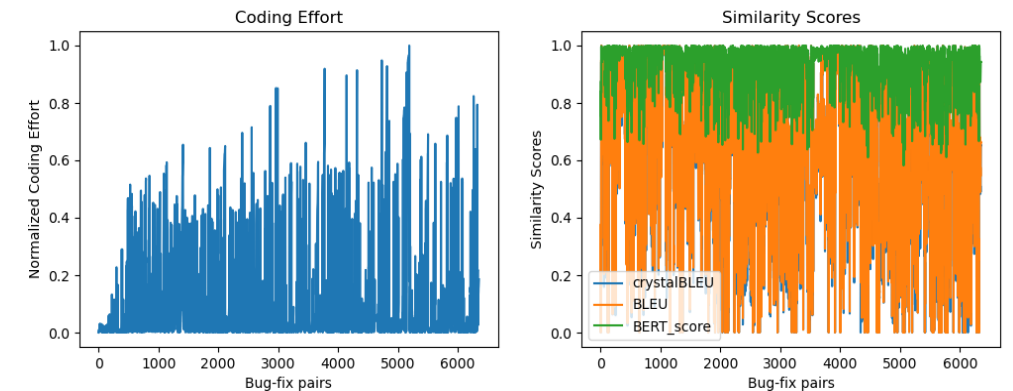
Evaluation

- MineCPP - run on **25 Python** projects (**#stars**, **#commits**)

- **Flask** – **61243 Stars** and **5096 commits**



- **Yt-dlp**– **35839 Stars** and **21800 commits**



- **Sawtooth** pattern observed in **Coding Effort** vs **Bug-fix** pairs across projects
 - due to **modifications** in import statements or modifications at the **beginning** of the file.

Dataset: 25 Python projects

- **Schema** of the dataset



<https://doi.org/10.5281/zenodo.10579446>

Col#	Name	Type	Description
1	Before Bug fix	String	Code + Context of Bug
2	After Bug fix	String	Code + Context of after Bug fix
3	Location	List of ints.	Line #s of the bug and its fix
4	Bug type	String	Bug type deduced using git diff and LLM
5	Commit Message	String	Author's description about commit
6	Project URL	String	GitHub link of the repository
7	File Path	String	The path of file with a change or bug
8	Fixed Commit	String	git hash of fixed commit
9	Buggy Commit	String	git hash of Buggy commit
10	Test File	Bool	has test case or not
11	Coding Effort	Int	explained in the CodingEffort function
12	Constructs	Dict	type of constructs in which bug is present
13	Lizard Features Buggy	Dict	Cyclomatic Complexity of buggy code
14	Lizard Features Fixed	Dict	Cyclomatic Complexity of fixed code
15	BLEU	Float	text similarity score
16	crystalBLEU_score	Float	code similarity score
17	BERT_score	Float	code similarity score

Schema of dataset

MineCPP

*"Overall I think this might be a great tool if it worked as advertised. While the setup and tool execution were **really smooth**, also thanks to the **simple interface**, the obtained output made me wonder if the submitted paper only tells half the story. It seems that the obtained data may entail significant and non-trivial post-processing to obtain an actual bug dataset."*

-Reviewer (FSE)

Downstream Usage

- **Researchers and Academics:** can utilize the dataset for empirical studies, data driven bug detection and auto-fix algorithms, and pattern mining-based bug fix algorithms.
- **Software Developers:** can leverage the dataset/tool for enhancing their code quality and bug fixing processes.
- **Educators and Students:** can incorporate the dataset/tool into their software engineering courses or research projects.

Future Scope

- **Bug type:** development of a more generalized bug type classification system that transcends project-specific distinctions.
- **Accuracy:** application of static/dynamic analysis to identify and eliminate false positives.
- **User study:** manual/semi-automatic inspection by developers needed to validate our datasets

Texts, References, and Acknowledgements

Online:

- Continuous Integration and Delivery (**CircleCI**: <https://circleci.com>)

Textbook:

- Sharp, J. (2022). *Microsoft Visual C# Step by Step*, 10th edition, Microsoft Press.
- Watson, K., Nagel, C., Pedersen, J. H., Reid, J. D., & Skinner, M. (2008). *Beginning Microsoft Visual C# 2008*. John Wiley & Sons.
- Mark J. Price (2024). *C# 13 and .NET 9 – Modern Cross-Platform Development Fundamentals*, 9th edition, Packt Publishing Ltd.

Reference:

- Soni, M. (2016). *DevOps for Web Development*. Packt Publishing Ltd.
- Yusuf Sulisty Nugroho, Hideaki Hata, and Kenichi Matsumoto. 2020. *How different are different diff algorithms in Git? Use --histogram for code changes*. Empirical Softw. Engg. 25, 1 (Jan 2020), 790–823.
- Sai Krishna Avula (2024), *Mining Software Bug Fixes and SAT Analysis: Dataset Creation and Tool Development for Improved Software Quality Assurance* - Awarded Gold Medal for the outstanding research (M.Tech.) at 13th Convocation IITGN.